

Representational Learning Assignment 2

By: Lucius Hatherly (20294880)

Problem 1:

Implementing a GRU Decoder with soft attention:

1. Refer to gru_solution.py for the solution.
2. Refer to the gru_solution.py for the solution.
3. Refer to the gru_solution.py for the solution.
4. Refer to the gru_solution.py for the solution.

Transformer Implementation:

1. Refer to transformer_solution.py for the solution.
2. Refer to transformer_solution.py for the solution.
3. Refer to transformer_solution.py for the solution.
4. Refer to transformer_solution.py for the solution.
5. Refer to transformer_solution.py for the code solution.
 - **Parameter count:** Let $m = \text{num_heads}$ and $d = \text{head_size}$. This gives the embedding dimension as $D = md$. The MultiHeaded Attention has four linear layers: 1. W_Q : $\text{nn.Linear}(D, D, \text{bias}=\text{True})$, 2. W_K : $\text{nn.Linear}(D, D, \text{bias}=\text{True})$, 3. W_V : $\text{nn.Linear}(D, D, \text{bias}=\text{True})$, and 4. W_Y : $\text{nn.Linear}(D, D, \text{bias}=\text{True})$. Note each layer contributes $D^2 + D$. With 4 layers this produces $4(D^2 + D)$. We can substitute $D = md$ and this produces $4(m^2d^2 + md)$. So the number of learnable parameters is $4m^2d^2 + 4md$. Equivalently, this is $4(md)(md+1)$.
6. Refer to transformer_solution.py for the code solution.
7. Refer to transformer_solution.py for the code solution.

Problem 2:

Q1.

1.

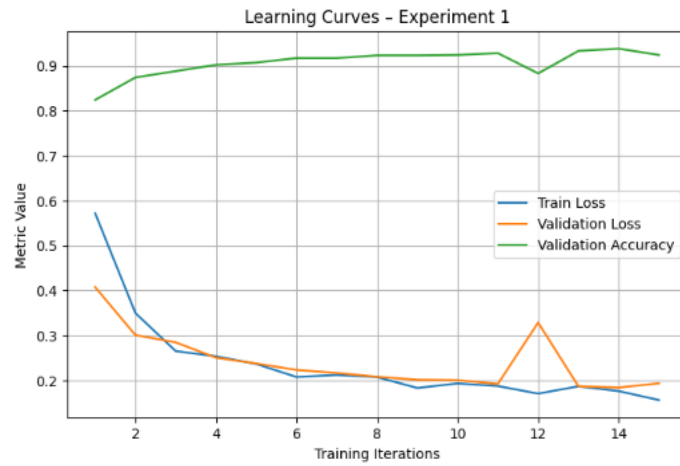


Figure: Learning curves showing training loss, validation loss, and validation accuracy over training iterations for Experiment 1.

(Curves for Experiment 1 - GRU: no dropout, encoder only)

2.

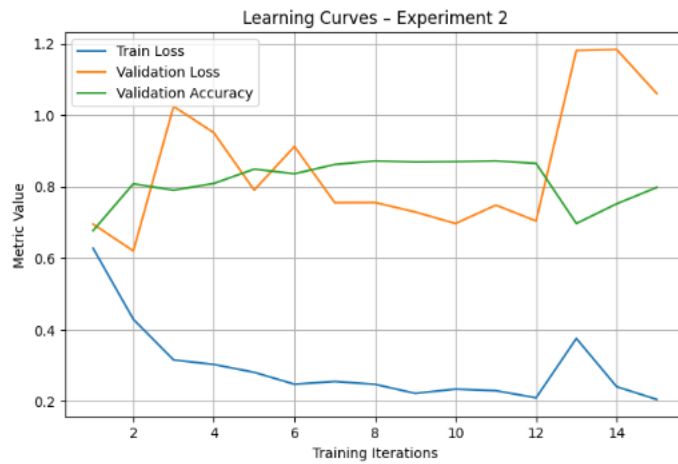


Figure: Learning curves showing training loss, validation loss, and validation accuracy over training iterations for Experiment 2.

(Curves for Experiment 2 - GRU dropout, encoder only)

3.

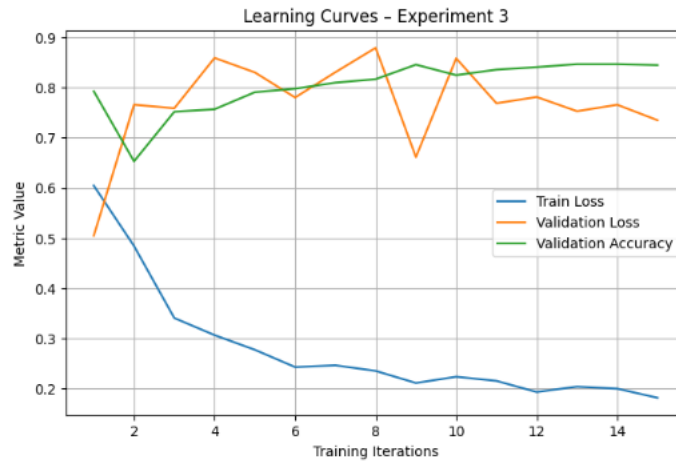


Figure: Learning curves showing training loss, validation loss, and validation accuracy over training iterations for Experiment 3.

(Curves for Experiment 3 - GRU: dropout, encoder-decoder, no attn)

4.

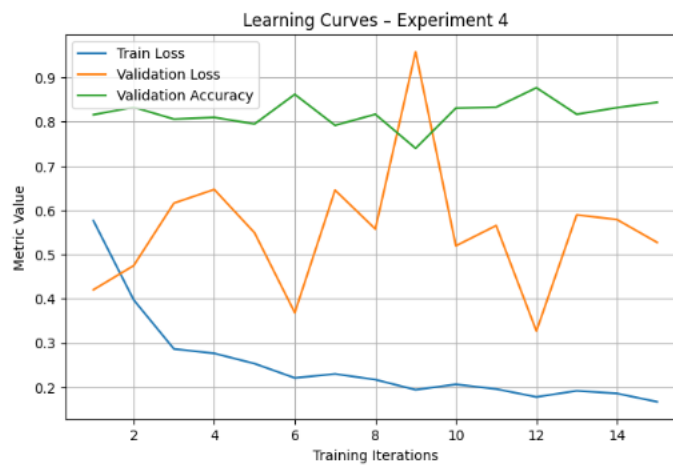


Figure: Learning curves showing training loss, validation loss, and validation accuracy over training iterations for Experiment 4.

(Curves for Experiment 4 - GRU: dropout, encoder-decoder, with attention)

5.

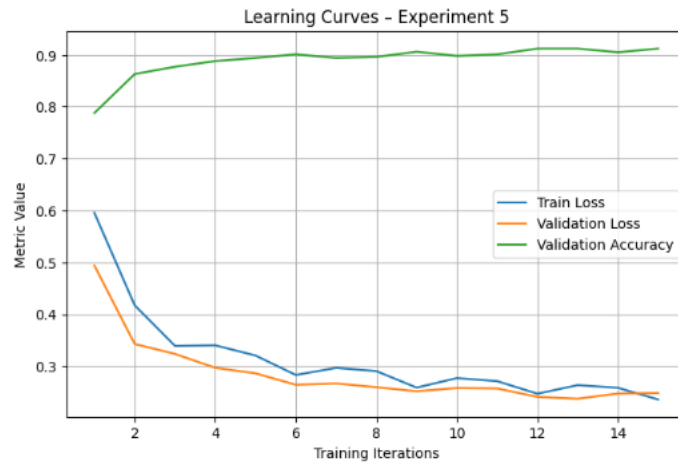


Figure: Learning curves showing training loss, validation loss, and validation accuracy over training iterations for Experiment 5.

(Curves for Experiment 5 - Transformer: 2 layers, prenorm)

6.

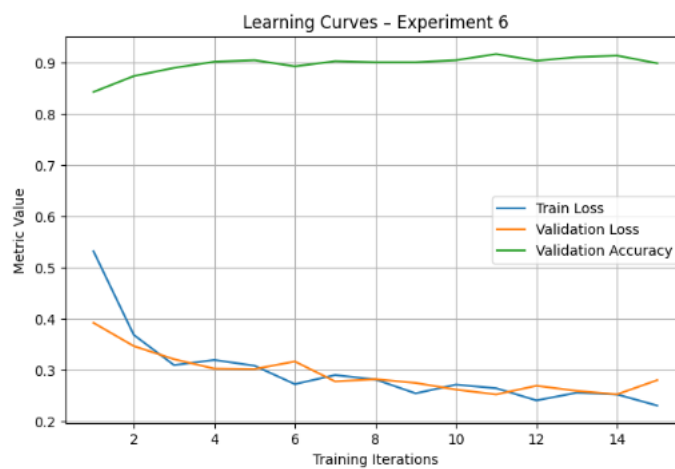


Figure: Learning curves showing training loss, validation loss, and validation accuracy over training iterations for Experiment 6.

(Curves for Experiment 6 - Transformer: 4 layers, prenorm)

7.

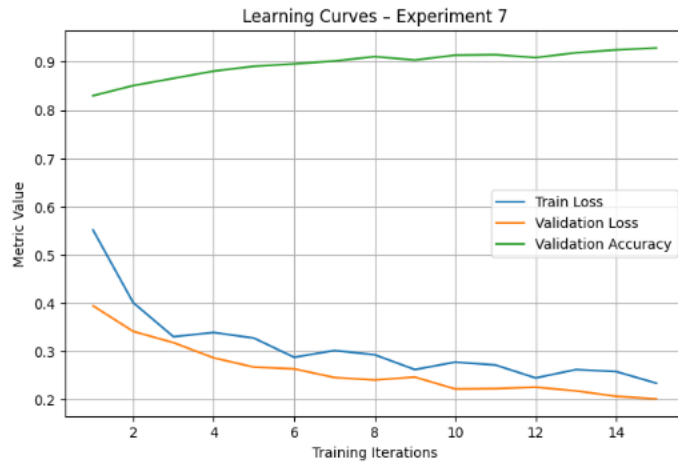


Figure: Learning curves showing training loss, validation loss, and validation accuracy over training iterations for Experiment 7.

(Curves for Experiment 7 - Transformer: 2 layers, postnorm)

8.

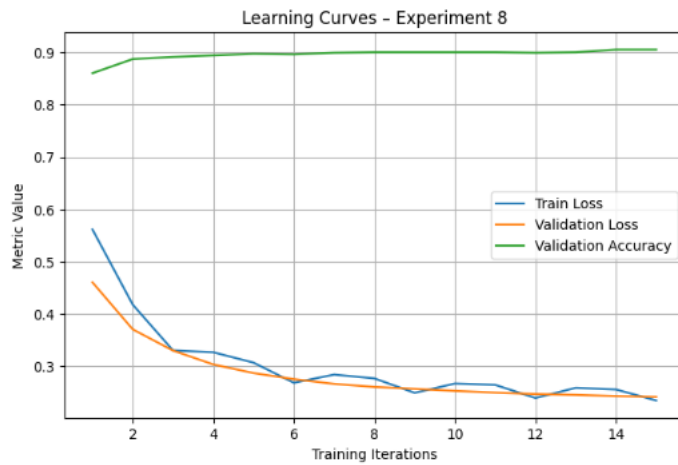


Figure: Learning curves showing training loss, validation loss, and validation accuracy over training iterations for Experiment 8.

(Curves for Experiment 8 -Pretrained BERT)

Q2.

Experiment Runtime Summary

Experiment	Architecture	Train Loss	Val Loss	Train Accuracy	Val Accuracy	Train Time (seconds)	Evaluation Time (seconds)	Total Time (seconds)
1	GRU: no dropout, encoder only	0.1780	0.1936	0.9353	0.938	980.57	1.47	982.05
2	GRU: dropout, encoder only	0.2812	1.0600	0.8089	0.872	979.71	1.48	981.19
3	GRU: dropout, encoder-decoder, no attention	0.2000	0.7350	0.8381	0.847	1061.86	1.54	1063.40
4	GRU: dropout, encoder-decoder, with attention	0.1865	0.5271	0.8514	0.877	1100.39	1.79	1102.18
5	Transformer: 2 layers, prenorm	0.2598	0.2485	0.9004	0.912	1547.83	1.78	1549.62
6	Transformer: 4 layers, prenorm	0.2534	0.2808	0.8940	0.917	2231.08	2.15	2233.23
7	Transformer: 2 layers, postnorm	0.2584	0.2016	0.9040	0.929	1564.22	1.79	1566.01
8	Pretrained BERT	0.2568	0.2418	0.8959	0.905	8512.28	14.21	8526.49

Table summarizing experiment results across 8 different architectures. Note Val Accuracy is Validation Accuracy and Val Loss is Validation Loss. We can see that GRU: no dropout encoder only is the best performing model for validation accuracy.

Q3. When the main concern is wall-clock time, Experiment 1 (GRU encoder only without dropout) is the best performing model and architecture type. It achieves a total runtime of approximately 980.57 seconds, which is among the fastest of all models tested. This is because the GRU encoder architecture is efficient computationally as it processes sequences sequentially without the complexity presented by multiple transformer layers or attention mechanisms. The table shows that the Transformer models (Experiments 5-7) need more computation because of self-attention operations seen across all tokens, and BERT pretrained models (Experiment 8) are much slower due to the high parameter count and comparatively deeper architecture. The GRU encoder architecture is computationally efficient because it processes sequentially without the additional complexity introduced by attention mechanisms or multiple transformer layers.

However, it should be noted that although Experiment 2 has a slightly faster training time than Experiment 1 (979.71 seconds vs 980.57 seconds), the less than one-second difference is thus insignificant. However, Experiment 1 provides much higher validation accuracy (0.938 vs 0.872) compared to Experiment 2. Thus, Experiment 1 gives the best balance when you analyze wallclock efficiency and predictive performance.

When you consider generalization performance as your primary concern, Experiment 1 should also be selected. This is because it achieves the highest validation accuracy of (0.938) across all the models. This result may appear surprising as pretrained BERT is generally expected to perform better due to the presence of large-scale pretraining. Although it achieves a lower validation accuracy of (0.905). This may be caused by limited data set size, hyperparameter settings, and finetuning epochs. In conclusion, Experiment 1 has the best performing validation accuracy compared to the seven other models and should be used when considering generalization performance.

Q4. The key difference between Experiments 1 and 2 is the presence of dropout regularization in the GRU encoder model for Experiment 2. The Dropout is normally used to minimize overfitting through randomly disabling a subset of neurons during training, which forces the model to learn a considerably more robust feature representation. However, we can see that adding a dropout causes a worse performance across validation metrics like training accuracy, training loss, validation accuracy, and validation loss. The training accuracy decreased from 0.935 to 0.809 while validation accuracy decreased from 0.938 to 0.872. Furthermore, the model's validation loss increased from 0.1936 to 1.060 which implies the model's performance was less stable for Experiment 2.

These results imply that the GRU model from Experiment 1 was not strongly overfitting, so that regularization introduced by the dropout disrupted the learning process. The dropout sometimes negatively impacts performance when the model is already simple or when you are dealing with a dataset too small to need stronger regularization. In these scenarios the GRU encoder already has sufficient capacity without overfitting, so including dropout layer reduces the model's ability to consistently learn patterns present within the training data.

Q5. We can see that Experiments 2 and 3 compare a GRU encoder model with a GRU encoder-decoder architecture. We see that the encoder-decoder model slightly improved training accuracy (0.838 vs. 0.809), but validation accuracy in fact declined (0.847 vs. 0.872). However, the training and validation loss both improved from Experiment 2 to Experiment 3 (0.2812 and 1.0600 vs 0.2000 and 0.7350 respectively). Overall, the decline in validation accuracy implies that the encoder-decoder architecture is not providing tangible benefits for this task and may have induced further complexity that reduced generalization.

We should note that Encoder-decoder architectures are generally constructed for sequence-to-sequence problems, including machine translation, summarization, or text generation, where the model generates an output sequence based on an input sequence. This task is for text classification, where the target is to map an input sequence to a single label. Usually, an encoder alone is enough to generate a contextual representation of the input sequence for a given classification task. Including a decoder will increase the parameter number and computational complexity without developing useful structure for the problem. This may cause the encoder-decoder architecture to slightly overfit training data while validation performance fails to improve.

Q6. Experiment 4 examines the impact of including soft attention to the GRU encoder-decoder architecture. When compared to Experiment 3, introducing attention improved both training and validation performance. We can see that the training accuracy increased from 0.838 to 0.851, while validation accuracy increased from 0.847 to 0.877, and validation loss decreased substantially from 0.735 to 0.527. These better results imply that the attention mechanism allows the model to focus better on the more important parts of the input sequence when its representation is being created.

The soft attention functions by allowing the model to compute weighted importance scores for each token in the sequence. This contrasts with compressing the entire sequence into a single hidden state. In recurrent architectures that exclude attention, the final hidden state summarizes the entire input sequence, which could cause important information to be lost, particularly for longer sequences. This contrasts with attention which allows the model to dynamically emphasize specific words or tokens most important for prediction. This tends to improve the model's ability to capture semantic relationships and long-range dependencies within the sequence. This likely explains the observed performance improvement.

Q7. The obtained results for experiments 5–8 somewhat align with expectations for Transformer-based models but the results also exhibit surprising behavior. Recently we have seen that Transformers and large pretrained models such as BERT have achieved state-of-the-art performance for many NLP tasks. This is because of their ability to capture long-range dependencies and contextual relationships via self-attention mechanisms. Across these experiments, the Transformer models (experiments 5–7) produced strong validation accuracies which lie between approximately 0.912 to 0.929. This confirms that architectures which use attention are effective for classifying text. However, we can see that the pretrained BERT model (experiment 8) does not outperform the simpler GRU encoder model and achieved a validation

accuracy of about 0.905. BERT is expected to perform the best due to its large-scale pretraining on massive corpora, several factors may explain this comparatively weaker performance seen in experiment 8.

This may be because the BERT model was only fine-tuned for a small number of epochs (5) and not wholly adapted to the task-specific dataset. Secondly, the experiment's dataset is rather small. This fact may allow smaller models like GRUs to perform competitively where there is no requirement for the representational power of large pretrained models. Transformer architecture also usually requires precise hyperparameter tuning and long training times to properly satisfy all their advantages. This results in the GRU baseline producing the best validation accuracy implying that simpler architectures can often outperform more complex models when the dataset size, training time, or tuning process is limited like we see across the experiments here.

Q8.

Experiment	Architecture	Estimated GPU Memory
1	GRU: no dropout, encoder only	8.69 GB
2	GRU: dropout, encoder only	8.75 GB
3	GRU: dropout, encoder-decoder, no attention	8.76 GB
4	GRU: dropout, encoder-decoder, with attention	9.24 GB
5	Transformer: 2 layers, prenorm	10.92 GB
6	Transformer: 4 layers, prenorm	20.25 GB
7	Transformer: 2 layers, postnorm	10.54 GB
8	Pretrained BERT	8.97 GB

The overall trend shows that GPU memory usage increases with the model's architecture's complexity. The GRU encoder-only models (Experiments 1 and 2) use the smallest amount of GPU memory, using approximately 8.69 GB and 8.75 GB respectively. These first two models process sequences sequentially and keep a single hidden state representation at each timestep. This enables the number of stored intermediate activations to be small. Experiment 2 does not impact memory usage significantly because dropout only masks activations through training and avoids large intermediate tensors or introducing more parameters.

The encoder-decoder GRU architectures (Experiments 3 and 4), increases GPU memory consumption to 8.76 GB and 9.24 GB. The increase is caused by the encoder-decoder structure having both additional hidden states for both the encoder and decoder networks during training. Experiment 4 also increases memory usage because the attention mechanisms increase memory consumption because attention matrices must be stored during forward and backward passes. produced. These additional intermediate tensors combined with the attention matrices require more memory usage compared to encoder-only GRU models.

The Transformer models (Experiments 5, 6, and 7) use more GPU memory where the usage is between approximately 10.92 GB to 20.25 GB. This higher usage level is mostly caused by the self-attention mechanism. This mechanism calculates the attention scores for all tokens within a sequence. This causes the memory complexity to increase approximately quadratically with respect to sequence length. Experiment 6 uses the most memory out of the transformer models as it uses 4 layers instead of 2. This causes more parameters and intermediate activations needed for backpropagation. We can see that Experiment 7 has similar memory usage to Experiment 5 because both models use two Transformer layers, and the difference between prenorm and postnorm normalization does not significantly alter the used memory footprint.

In conclusion, the pretrained BERT model (Experiment 8) has a GPU memory footprint at around 8.97 GB. This is surprising, as we should expect a larger if not the largest memory usage because BERT is a large pretrained Transformer model with many layers, a large embedding matrix, and requires a significantly higher number of parameters compared to other models in the assignment. One reason why the pretrained BERT model uses less memory than expected is because highly optimized implementations in libraries like HuggingFace Transformers efficiently manage the storage of parameters and operations involving attention. Secondly, the small sequence length and batch sizes further put constraints on how much memory is used.

Essentially, more complex architectures, parameters, layers, and networks will need more GPU usage in comparison to simpler models.

Q9. Overall, across all the 8 Learning Curves we see these two consistent trends: 1) Training loss consistently decreases across the iterations, 2) Validation accuracy consistently increases before plateauing (with some fluctuation for Experiments 2 to 4). However, Experiments 1, 5, 6, 7, 8 (the GRU baseline, Transformer, and Pretrained models) showed curves which consistently decreased the validation loss before plateauing. While Experiments 2 to 4 showed Validation Loss Curves which fluctuated drastically.

The increased regularization and architectural complexity introduced in these models compared to the baseline GRU encoder could explain the comparatively higher and fluctuating validation loss. In Experiment 2, dropout introduces stochastic noise during training by randomly deactivating neurons, which can make validation loss fluctuate more even though it helps prevent overfitting. In Experiments 3 and 4, the encoder-decoder architecture adds additional parameters and intermediate states. This serves to increase the model complexity for a task that only requires sequence classification. This additional capacity could make optimization less stable and cause larger fluctuations in validation loss. Note that this is especially possible when the dataset is relatively small. Furthermore, the attention mechanism in Experiment 4 will produce additional attention weight matrices that serve to change dynamically during training. This can also contribute to noisier validation loss curves. These models will then show less smooth validation loss behavior compared to the simpler encoder-only GRU and Transformer models.

Simple models like the GRU encoder (Experiment 1) show stable and smooth learning behavior, with validation accuracy quickly converging to a high value along with the validation loss decreasing

drastically. The model possessed enough capacity to minimize overfitting and properly learn patterns. Experiment 2 saw the dropout being added and training became more unstable along with the training and validation performance also decreasing compared to Experiment 1. This suggests that the model from Experiment 1 was not strongly overfitted and further did not improve because of the additional regularization added in Experiment 2. Furthermore, Experiment 2's validation loss was much more unstable than Experiment 1, it increased overall while fluctuating widely, compared to the steady decrease of Experiment 1. The GRU encoder-decoder architecture (Experiment 3) demonstrated lower validation accuracy, which further illustrates possible mild overfitting or unnecessary model complexity with respect to classification task. In addition, Experiment 3 Validation Loss curve followed the same trends as Experiment 2 by increasing overall with significant fluctuations.

We can see that the Transformer-based models demonstrate different learning behaviors because of their increased representational capacity and need for attention mechanisms. These models mostly produced strong validation accuracy but needed much longer training times and exhibited marginally noisier validation loss curves. The pretrained BERT model also demonstrated stable training but could not outperform the GRU baseline from Experiment 1. This could indicate limited underfitting because of a lack of insufficient task-specific adaptation.

In terms of overfitting and underfitting, the simpler GRU encoder model (Experiment 1) demonstrated the best balance between model capacity and generalization, as the training and validation curves remained close and stable throughout training. The encoder-decoder architectures (Experiments 3 and 4) introduced additional model capacity that was unnecessary for this classification task, which may have contributed to mild overfitting and greater instability in validation loss. In contrast, the transformer-based models and the pretrained BERT model showed slightly lower improvements relative to their capacity, suggesting mild underfitting due to limited training epochs or insufficient task-specific fine-tuning. We can see that practitioners are able to reduce overfitting or instability by modifying hyperparameters like dropout rate, learning rate, model depth, and batch size. Finally, more techniques like gradient clipping, early stopping and longer fine-tuning schedules further help stabilize training and better the generalization performance.

Carrying on, as mentioned before, hyperparameters and architectural configurations are other key factors influencing learning curves across experiments. Across various experiments hyperparameters changed included dropout rate, model depth, attention mechanisms, and normalization strategies. We can see that Experiment 2 introduced dropout compared to Experiment 1, which acts as a regularization technique through masking activation randomly during training. For this example, we saw that dropout slightly reduced both training and validation performance. This implies that the original model was not significantly overfitting and further did not benefit from more regularization. We can see that Experiments 5, 6, and 7 further explored multiple Transformer configurations through changing the number of layers from two to four and pre or post normalization. We saw that increasing the number of layers from two to four increased the model capacity but also saw an increase in parameters and the computational cost. We additionally see that the difference between prenorm and postnorm normalization impacted the

training convergence and stability behavior. All these results further illustrate how architectural and hyperparameter choices like dropout, model depth, attention mechanisms, and normalization placement all impact convergence, learning rate, and generalization.

Q10. We can define an attention mechanism as a neural network technique which permits a model to concentrate on the most pertinent parts of an input sequence when this model is making predictions. This contrasts with compressing an entire sequence into a single fixed-length representation, attention instead calculates a set of weights which highlights the importance of each token in the sequence is compared to a particular query. These weights can then be combined to generate a weighted combination of value vectors. This permits the model to selectively highlight important information. We can say that Attention mechanisms are very useful in natural language processing (NLP) tasks because we can see that important contextual information could appear at any place in a sequence. The attention helps the model in capturing long-range dependencies while we avoid exclusively focusing on sequential processing like GRU models.

Secondly, Self-attention and cross-attention are two related types of attention regularly used in modern neural architectures. For self-attention, they have queries, keys, and values which are all obtained from the same sequence. This allows each token to attend to every other token located in the sequence. This specific mechanism is the key component of Transformer architecture and allows models to capture relationships between the words regardless of the distance to the input. This is different to cross-attention in which the queries come from one sequence while the keys and values are obtained from a separate sequence. This cross-attention is usually used in encoder-decoder architecture where we see the decoder attending to the encoded representation for the given input sequence. We see both mechanisms relying on the same calculated attention, self-attention models internalize relationships within a given sequence, where we can see that cross-attention permits the information flow between different sequences.